

## **Bachelorarbeits Skizze**

# **Automatisierte Validierung von RAG-Agents: Konzeption eines Frameworks zur Prüfung von Antwortqualität und Sicherheit durch Adversarial Agents**

|                                |                              |
|--------------------------------|------------------------------|
| <b>Name:</b>                   | Marko Perkovic               |
| <b>Matrikel-Nummer:</b>        | 1147137                      |
| <b>Kurs:</b>                   | WWI23SCB                     |
| <b>Ausbildungsunternehmen:</b> | Syntax Systems GmbH & Co. KG |
| <b>Datum der Abgabe:</b>       | 10.02.2026                   |

# Inhaltsverzeichnis

## Inhalt

|                                                 |    |
|-------------------------------------------------|----|
| Inhaltsverzeichnis.....                         | 2  |
| 1. Einleitung.....                              | 3  |
| 1.1 Motivation.....                             | 3  |
| 1.2 Ziel der Projektskizze.....                 | 3  |
| 2. Forschungsfragen der Bachelorarbeit.....     | 4  |
| 3. Methodik.....                                | 4  |
| 3.1 Design Science Research.....                | 4  |
| 3.2 Evaluationsmetriken.....                    | 5  |
| 4. Aufbau der Bachelorarbeit.....               | 6  |
| 5. Grundlagen.....                              | 6  |
| 5.1 Retrieval Augmented Generation (RAG).....   | 6  |
| 5.2 Context Hallucination und Faithfulness..... | 7  |
| 5.3 Indirect Prompt Injection.....              | 7  |
| 5.4 LLM-as-a-Judge.....                         | 7  |
| 6. Praktische Skizze.....                       | 8  |
| 6.1 Architektur des Validierungsframeworks..... | 8  |
| 6.2 Adversarial Test Case Generation.....       | 9  |
| 6.3 CI/CD-Integration.....                      | 9  |
| 7. Zeitplanung.....                             | 10 |
| 8. Schlussbetrachtung.....                      | 10 |
| 8.1 Einordnung der Skizze.....                  | 10 |
| 8.2 Limitationen.....                           | 11 |
| 8.3 Ausblick.....                               | 11 |
| i. Literaturverzeichnis.....                    | 11 |

# 1. Einleitung

## 1.1 Motivation

Der Einsatz von generativer KI in Unternehmensprozessen verlagert sich zunehmend auf Retrieval Augmented Generation (RAG). Diese Agenten generieren Antworten nicht nur aus ihrem Trainingswissen, sondern greifen dynamisch auf interne Wissensdatenbanken zu, um fundierte Entscheidungen zu treffen. Mit dieser Anbindung an externe Daten steigen jedoch die Risiken erheblich.

Ein zentrales Problem ist die "Context Hallucination", bei der der Agent trotz korrekter Dokumente falsche Antworten generiert (mangelnde Faithfulness). Aktuelle Forschung zeigt, dass selbst modernste LLMs häufig ungestützte Informationen oder Widersprüche einführen, obwohl ihnen relevanter Kontext zur Verfügung steht [1]. Zudem eröffnet die Verarbeitung externer Daten neue Angriffsvektoren wie die "Indirect Prompt Injection", bei der Angreifer bösartige Befehle in Dokumenten verstecken, die der Agent unwissentlich verarbeitet. Studien belegen, dass bereits fünf sorgfältig konstruierte Dokumente genügen können, um RAG-Systeme in 90% der Fälle zu manipulierten Antworten zu verleiten [2].

Klassische Softwaretests greifen hier nicht, da RAG-Systeme nicht deterministisch arbeiten. Es fehlt an etablierten Prozessen im Application Lifecycle Management (ALM), um sicherzustellen, dass ein deployter RAG-Agent dauerhaft sicher und faktentreu agiert. Die manuelle Prüfung ist nicht skalierbar. Daher gewinnen automatisierte Ansätze, bei denen ein LLM (als "Evaluator") die Retrieval-Qualität und Sicherheit prüft, massiv an Bedeutung.

## 1.2 Ziel der Projektskizze

Diese Skizze definiert den Rahmen für die Bachelorarbeit. Sie umfasst die Konkretisierung der Problemstellung (Retrieval-Qualität vs. Security), die Ableitung der Forschungsfragen sowie die methodische Planung eines Validierungs-Frameworks. Ziel ist es, ein Konzept zu entwickeln, das den manuellen Testaufwand durch den Einsatz von "Adversarial Agents" automatisiert und so die Robustheit produktiver RAG-Anwendungen sicherstellt.

Die Skizze dokumentiert darüber hinaus das geplante Vorgehen und umfasst die Wahl und Konkretisierung des Themas, die Formulierung der Problemstellung und Ziele, die Darstellung der Literatur- und Informationsbasis sowie die Beschreibung des methodischen Vorgehens. Ergänzend wird die geplante Gliederung der Arbeit sowie ein detaillierter Zeitplan präsentiert, um die Durchführbarkeit des Projekts innerhalb des vorgegebenen Zeitrahmens sicherzustellen.

## 2. Forschungsfragen der Bachelorarbeit

Die Arbeit fokussiert sich auf die Schnittstelle zwischen Qualitätssicherung und IT-Security bei generativer KI. Es werden zwei zentrale Forschungsfragen untersucht:

**Forschungsfrage 1:** Wie muss ein Validierungsframework konzipiert sein, um RAG-Agenten automatisiert auf Context Hallucination und Indirect Prompt Injection zu prüfen?

Diese Frage zielt auf die technische Konzeption ab. Es soll untersucht werden, wie eine Architektur aussehen muss, in der ein "Adversarial Agent" (Angreifer/Prüfer) gezielt Schwachstellen im Retrieval- und Generierungsprozess aufdeckt. Dabei werden sowohl die Generierung adversarialer Testfälle als auch die systematische Bewertung der Ergebnisse betrachtet.

**Forschungsfrage 2:** Inwieweit eignet sich der "LLM-as-a-Judge"-Ansatz zur zuverlässigen Bewertung von RAG-spezifischen Metriken (wie Faithfulness und Answer Relevance) in einem Continuous-Deployment-Prozess?

Hier steht die Evaluation der Methodik im Vordergrund. Es wird analysiert, wie verlässlich ein KI-Modell die Qualität der Quellenverarbeitung bewerten kann und ob dieser Ansatz in eine CI/CD-Pipeline integrierbar ist, um das ALM von RAG-Agenten abzusichern. Aktuelle Forschung zeigt, dass LLM-Judges mit menschlichen Präferenzen in über 80% der Fälle übereinstimmen können [3], jedoch diverse Biases wie Position Bias, Verbosity Bias und Self-Enhancement Bias aufweisen [4].

## 3. Methodik

### 3.1 Design Science Research

Design Science Research (DSR) ist ein etabliertes Forschungsparadigma in der Wirtschaftsinformatik, das sich besonders für die Entwicklung innovativer IT-Artefakte eignet [5]. Im Gegensatz zum verhaltenswissenschaftlichen Paradigma, das Theorien entwickelt und verifiziert, zielt DSR darauf ab, die Grenzen menschlicher und organisatorischer Fähigkeiten durch die Schaffung neuer Artefakte zu erweitern.

Diese Arbeit folgt den sieben DSR-Guidelines nach Hevner et al. [5]:

- **Design as an Artifact:** Entwicklung eines Validierungsframeworks als konkretes IT-Artefakt
- **Problem Relevance:** Adressierung des praktischen Problems fehlender automatisierter RAG-Validierung
- **Design Evaluation:** Rigorose Bewertung durch empirische Experimente mit adversarialen Testfällen

- **Research Contributions:** Beitrag sowohl zum Artefakt-Wissen als auch zur methodischen Wissensbasis
- **Research Rigor:** Anwendung rigoroser Methoden bei Konstruktion und Evaluation
- **Design as a Search Process:** Iterative Build-and-Evaluate-Zyklen
- **Communication of Research:** Dokumentation für technologie- und managementorientierte Zielgruppen

Der konkrete DSR-Prozess folgt dem Modell nach Peffers et al. [6] mit den Phasen: Problem Identification, Objectives Definition, Design & Development, Demonstration, Evaluation und Communication.

### 3.2 Evaluationsmetriken

Zur Bewertung der RAG-Qualität werden etablierte Metriken aus dem RAGAS-Framework (Retrieval Augmented Generation Assessment) herangezogen [1]. RAGAS bietet eine Suite von Metriken, die ohne Ground-Truth-Annotationen auskommen und somit für automatisierte Evaluierung geeignet sind.

#### Retrieval-Metriken:

- **Context Precision:** Misst das Signal-Rausch-Verhältnis des abgerufenen Kontexts
- **Context Recall:** Prüft, ob alle relevanten Informationen für die Beantwortung der Frage abgerufen wurden
- **Context Relevance:** Bewertet, ob der abgerufene Kontext minimal irrelevante Informationen enthält

#### Generator-Metriken:

- **Faithfulness:** Misst die faktische Konsistenz der generierten Antwort gegenüber dem abgerufenen Kontext. Berechnet als Verhältnis der durch den Kontext gestützten Aussagen zur Gesamtzahl der Aussagen
- **Answer Relevance:** Quantifiziert, wie relevant die generierte Antwort für die gestellte Frage ist
- **Answer Correctness:** Kombiniert semantische Ähnlichkeit mit faktischer Korrektheit

#### Security-Metriken:

- **Attack Success Rate (ASR):** Anteil erfolgreicher Prompt-Injection-Angriffe
- **Instruction Override Rate:** Rate, mit der System-Instruktionen durch adversariale Eingaben überschrieben werden
- **Data Exfiltration Success:** Erfolgsrate bei Versuchen, sensible Daten zu extrahieren

## 4. Aufbau der Bachelorarbeit

| Kapitel | Abschnitt            | Unterabschnitt                                                                    |
|---------|----------------------|-----------------------------------------------------------------------------------|
| 1       | Einleitung           | Motivation, Ziel der Arbeit, Aufbau                                               |
| 2       | Grundlagen           | RAG-Architektur, Context Hallucination, Indirect Prompt Injection, LLM-as-a-Judge |
| 3       | Methodik             | Design Science Research, RAGAS-Metriken, Evaluationsdesign                        |
| 4       | Framework-Konzeption | Architektur, Adversarial Agent Design, Testfall-Generierung, CI/CD-Integration    |
| 5       | Implementierung      | Prototypische Umsetzung, Technologie-Stack, Code-Beispiele                        |
| 6       | Evaluation           | Experimentdesign, Ergebnisse, Hypothesentests, Vergleich mit Baseline             |
| 7       | Schlussbetrachtung   | Zusammenfassung, Einordnung, Limitationen, Ausblick                               |

*Tabelle 1: Geplante Gliederung der Bachelorarbeit*

## 5. Grundlagen

### 5.1 Retrieval Augmented Generation (RAG)

Retrieval Augmented Generation (RAG) ist ein Architekturparadigma, das Large Language Models (LLMs) mit externen Wissensquellen verbindet [7]. Die Grundidee besteht darin, die generativen Fähigkeiten von LLMs mit einer Retrieval-Komponente zu kombinieren, die relevante Dokumente aus einer Wissensdatenbank abrufen. Dadurch können LLMs auf aktuelle, domänenspezifische oder proprietäre Informationen zugreifen, die nicht in ihrem Trainingsdatensatz enthalten waren.

Eine typische RAG-Pipeline besteht aus drei Hauptkomponenten: dem **Retriever**, der relevante Dokumente basierend auf der Nutzeranfrage aus einem Vektorindex abrufen; dem **Context Assembler**, der die abgerufenen Dokumente mit der ursprünglichen Anfrage zu einem erweiterten Prompt zusammenführt; und dem **Generator** (LLM), der basierend auf dem erweiterten Kontext eine Antwort generiert.

RAG-Systeme bieten gegenüber reinen LLM-Anwendungen mehrere Vorteile: Sie reduzieren das Risiko von Halluzinationen durch Grounding in faktischen Dokumenten, ermöglichen den Zugriff auf aktuelles Wissen jenseits des Knowledge Cutoffs, und erlauben die Integration proprietärer Unternehmensdaten. Gleichzeitig führt die Komplexität der Pipeline zu neuen Herausforderungen in den Bereichen Qualitätssicherung und Sicherheit.

## 5.2 Context Hallucination und Faithfulness

Context Hallucination, auch als Faithfulness Hallucination bezeichnet, beschreibt das Phänomen, bei dem ein LLM Antworten generiert, die nicht durch den bereitgestellten Kontext gestützt werden [8]. Im Gegensatz zur Factuality Hallucination, bei der generierter Inhalt von realen Weltfakten abweicht, bezieht sich Faithfulness Hallucination auf die Inkonsistenz zwischen generierter Antwort und dem zur Verfügung gestellten Kontext.

Die Ursachen für Context Hallucination sind vielfältig: Aufmerksamkeitsdefizite bei langen Kontexten führen dazu, dass LLMs lokale Informationen bevorzugen und entfernte relevante Passagen übersehen. Konflikte zwischen parametrischem Wissen (aus dem Training) und kontextuellem Wissen (aus RAG) können zu inkonsistenten Ausgaben führen. Zudem neigen LLMs dazu, übermäßig konfidente Antworten zu generieren, selbst wenn der Kontext unzureichend oder widersprüchlich ist [8].

Die Messung von Faithfulness erfolgt typischerweise durch Natural Language Inference (NLI)-basierte Methoden, die prüfen, ob jede Aussage der generierten Antwort durch den Kontext entailed (gestützt) wird. Das RAGAS-Framework [1] zerlegt Antworten in atomare Behauptungen und verifiziert diese einzeln gegen den Kontext, um einen Faithfulness-Score zwischen 0 und 1 zu berechnen.

## 5.3 Indirect Prompt Injection

Indirect Prompt Injection (IPI) stellt einen der kritischsten Sicherheitsrisiken für RAG-Systeme dar und wird von OWASP als Top-1-Risiko für LLM-Anwendungen eingestuft [9]. Im Gegensatz zur direkten Prompt Injection, bei der ein Angreifer bösartige Eingaben direkt an das LLM sendet, erfolgt IPI durch das Einschleusen von Anweisungen in externe Datenquellen, die vom RAG-System verarbeitet werden.

Der Angriffsvektor funktioniert folgendermaßen: Ein Angreifer platziert versteckte Instruktionen in Dokumenten, die in die Wissensdatenbank des RAG-Systems aufgenommen werden. Wenn diese Dokumente bei einer Nutzeranfrage abgerufen werden, interpretiert das LLM die eingebetteten Anweisungen als Teil des legitimen Kontexts und führt sie aus. Forschung zeigt, dass bereits fünf sorgfältig konstruierte Dokumente in einem Corpus von Millionen ausreichen, um Antworten in 90% der Fälle zu manipulieren [2].

Konkrete Angriffsszenarien umfassen: - **Datenexfiltration:** Manipulation des LLMs zur Preisgabe sensibler Informationen - **Instruction Override:** System-Prompts werden durch injizierte Anweisungen überschrieben - **Cross-Context Contamination:** Ausbreitung in Multi-Agent-Systemen - **Denial-of-Service:** "Blocker Documents" sabotieren gezielte Anfragen [10]

## 5.4 LLM-as-a-Judge

Der LLM-as-a-Judge-Ansatz nutzt Large Language Models als automatisierte Evaluatoren für die Bewertung von Text- und Modellausgaben [3]. Dieser Ansatz hat sich als skalierbare Alternative zu menschlicher Bewertung etabliert, insbesondere für Aufgaben, bei denen traditionelle Metriken wie BLEU oder ROUGE die Qualität nur unzureichend erfassen können.

Studien zeigen, dass starke LLM-Judges wie GPT-4 in über 80% der Pairwise-Vergleiche mit menschlichen Präferenzen übereinstimmen [11]. Gleichzeitig weisen LLM-Judges systematische Biases auf: - **Position Bias:** Bevorzugung von Antworten an bestimmten Positionen - **Verbosity Bias:** Tendenz zu längeren Antworten - **Self-Enhancement Bias:** Bevorzugung von Ausgaben des eigenen Modells - **Authority Bias:** Beeinflussung durch vermeintliche Expertise [4]

Für die Evaluation von RAG-Systemen ist der LLM-as-a-Judge-Ansatz besonders relevant, da er die Bewertung komplexer, domänenspezifischer Antworten ohne vorherige Ground-Truth-Annotationen ermöglicht. Aktuelle Best Practices umfassen: Few-Shot-Prompting mit Beispielen; explizite Rubrics und Score-Beschreibungen; Chain-of-Thought-Reasoning im Evaluationsprozess; und Position-Swapping zur Mitigation von Position Bias [3].

## 6. Praktische Skizze

Die praktische Umsetzung des Validierungsframeworks wird entlang von drei Kernkomponenten konzipiert, die gemeinsam eine automatisierte End-to-End-Validierung von RAG-Systemen ermöglichen.

### 6.1 Architektur des Validierungsframeworks

Das Framework folgt einer mehrschichtigen Verteidigungsarchitektur (Defense-in-Depth), die sowohl Qualitäts- als auch Sicherheitsaspekte abdeckt. Die Architektur besteht aus folgenden Komponenten:

**Test Case Generator:** Generiert automatisiert adversariale Testfälle für verschiedene Angriffskategorien (Direct Injection, Context Manipulation, Instruction Override, Data Exfiltration, Cross-Context Contamination). Basiert auf Template-basierten und LLM-generierten Ansätzen.

**RAG Pipeline Wrapper:** Abstraktionsschicht, die beliebige RAG-Implementierungen anbinden kann und standardisierte Schnittstellen für Retrieval, Context Assembly und Generation bereitstellt. Ermöglicht Logging aller Zwischenergebnisse.

**LLM-Judge-Modul:** Implementiert die Evaluation mittels RAGAS-Metriken (Faithfulness, Answer Relevance, Context Precision/Recall) sowie Security-spezifische Prüfungen. Unterstützt multiple Judge-Modelle für Ensemble-Bewertungen.

**Content Filter:** Embedding-basierte Anomalie-Erkennung zur Identifikation potenziell bösartiger Eingaben vor der Verarbeitung durch das RAG-System.

**Response Verifier:** Post-hoc-Konsistenzprüfung der generierten Antworten auf Anzeichen erfolgreicher Injections oder Halluzinationen.

Die Architektur orientiert sich an aktueller Forschung [10], die zeigt, dass kombinierte Verteidigungsstrategien die Erfolgsrate von Angriffen von 73,2% auf 8,7% reduzieren können, während 94,3% der Baseline-Task-Performance erhalten bleiben.



## 6.2 Adversarial Test Case Generation

Die Generierung adversarialer Testfälle erfolgt kategorisiert nach Angriffstypen und Komplexitätsstufen:

**Kategorie 1 - Faithfulness-Testfälle:** Fragen, deren korrekte Antwort explizit im Kontext enthalten ist, kombiniert mit ablenkenden Dokumenten. Prüft, ob das LLM dem Kontext treu bleibt oder auf parametrisches Wissen zurückgreift.

**Kategorie 2 - Context Manipulation:** Dokumente mit widersprüchlichen Informationen zur selben Entität. Prüft das Verhalten bei Konflikten zwischen abgerufenen Dokumenten.

**Kategorie 3 - Direct Injection:** Nutzeranfragen mit eingebetteten Anweisungen ("Ignore previous instructions..."). Prüft die Robustheit des System-Prompts.

**Kategorie 4 - Corpus Poisoning:** Optimierte Dokumente, die bei bestimmten Queries mit hoher Wahrscheinlichkeit abgerufen werden und versteckte Instruktionen enthalten.

**Kategorie 5 - Data Exfiltration:** Versuche, das LLM zur Preisgabe von System-Prompts, Kontext-Dokumenten oder sensiblen Informationen zu verleiten.

Für jede Kategorie werden sowohl template-basierte Testfälle (deterministisch, reproduzierbar) als auch LLM-generierte Varianten (diverse, adaptive) erstellt. Die Gesamtzahl der Testfälle orientiert sich an Benchmarks wie [10] mit ca. 800-1000 Testfällen über alle Kategorien.

## 6.3 CI/CD-Integration

Ein zentrales Ziel der Arbeit ist die Integration des Validierungsframeworks in bestehende CI/CD-Pipelines, um kontinuierliche Qualitäts- und Sicherheitsprüfungen bei RAG-Deployments zu ermöglichen.

- **Pre-Deployment Validation:** Vollständiger Testlauf gegen alle adversarialen Kategorien vor jedem Deployment. Definition von Quality Gates mit Schwellwerten für Faithfulness ( $\geq 0.85$ ), ASR ( $\leq 0.10$ ), etc.
- **Continuous Monitoring:** Stichprobenartige Evaluation von Produktions-Queries mit automatischer Alerting bei Metrik-Degradation
- **Regression Testing:** Automatisierte Tests bei Knowledge-Base-Updates, um unbeabsichtigte Qualitätseinbußen zu erkennen
- **Feedback Loop:** Aggregation von Evaluationsergebnissen zur kontinuierlichen Verbesserung der Testfall-Bibliothek

## 7. Zeitplanung

| Zeitraum         | Aktivitäten                                                                                                   | Meilensteine                                   |
|------------------|---------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| <b>Woche 1-2</b> | Literaturrecherche vertiefen, Related Work analysieren, Anforderungskatalog erstellen, Architekturentwurf     | M1: Anforderungen und Architektur spezifiziert |
| <b>Woche 3-4</b> | Implementierung Test Case Generator, RAGAS-Integration, Erste Prototyp-Version; Schreiben: Grundlagen-Kapitel | M2: Prototyp v0.1 lauffähig                    |
| <b>Woche 5-6</b> | LLM-Judge-Modul implementieren, Adversarial Test Cases erweitern, Content Filter; Schreiben: Methodik-Kapitel | M3: Vollständiges Framework                    |
| <b>Woche 7</b>   | Experimentdesign finalisieren, Baseline-Experimente durchführen; Schreiben: Framework-Konzeption              |                                                |
| <b>Woche 8</b>   | Vollständige Evaluation durchführen, Ergebnisse analysieren, Hypothesen testen; Schreiben: Implementierung    | M4: Evaluation abgeschlossen                   |
| <b>Woche 9</b>   | Schreibphase: Evaluation-Kapitel, Schlussbetrachtung; Abbildungen und Anhang                                  | M5: Hauptteil fertig                           |
| <b>Woche 10</b>  | Review, Konsistenzprüfung, Korrektorat, finale Formatierung; Abgabevorbereitung                               | M6: Abgabe                                     |

*Tabelle 2: Zeitplan der Bachelorarbeit (10 Wochen)*

## 8. Schlussbetrachtung

### 8.1 Einordnung der Skizze

Die vorliegende Skizze definiert einen strukturierten Rahmen für die Konzeption eines automatisierten Validierungsframeworks für RAG-Systeme. Der gewählte Design-Science-Research-Ansatz ermöglicht die rigorose Entwicklung und Evaluation eines praxisrelevanten IT-Artefakts, das sowohl Qualitäts- als auch Sicherheitsaspekte adressiert.

Die Kombination aus RAGAS-basierten Qualitätsmetriken und adversarialen Sicherheitstests schließt eine wichtige Lücke im Application Lifecycle Management von RAG-Anwendungen. Die geplante CI/CD-Integration stellt sicher, dass die Validierung nicht nur einmalig, sondern kontinuierlich erfolgen kann.

## 8.2 Limitationen

Die Arbeit fokussiert sich auf textbasierte RAG-Systeme. Multimodale Agenten mit Bild-, Audio- oder strukturierten Daten erfordern zusätzliche Angriffstypen und Evaluationsmetriken. Zudem basiert die Evaluation auf dem LLM-as-a-Judge-Ansatz, dessen inhärente Biases die Messung verzerren können. Die Generalisierbarkeit der Ergebnisse auf verschiedene LLM-Architekturen und Domänen muss empirisch validiert werden.

## 8.3 Ausblick

Zukünftige Erweiterungen könnten die Integration multimodaler Angriffsvektoren, die Anwendung auf Agentic AI-Systeme mit Tool-Use, sowie die Entwicklung adaptiver Verteidigungsmechanismen umfassen, die sich automatisch an neue Angriffstypen anpassen. Auch die Erforschung von Red-Teaming-Ansätzen, bei denen LLMs automatisiert neue Angriffsstrategien entwickeln, bietet vielversprechende Forschungsrichtungen.

## i. Literaturverzeichnis

- [1] S. Es, J. James, L. Espinosa Anke, und S. Schockaert, “RAGAs: Automated Evaluation of Retrieval Augmented Generation”, in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024, S. 150–158. doi: 10.18653/v1/2024.eacl-demo.16.
- [2] W. Zou et al., “PoisonedRAG: Knowledge Poisoning Attacks to Retrieval-Augmented Generation of Large Language Models”, *arXiv preprint arXiv:2402.07867*, 2024.
- [3] J. Gu et al., “A Survey on LLM-as-a-Judge”, *arXiv preprint arXiv:2411.15594*, 2024.
- [4] G. H. Chen, S. Chen, Z. Liu, F. Jiang, und B. Wang, “Humans or LLMs as the Judge? A Study on Judgement Bias”, in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024. doi: 10.18653/v1/2024.emnlp-main.474.
- [5] A. R. Hevner, S. T. March, J. Park, und S. Ram, “Design Science in Information Systems Research”, *MIS Quarterly*, Bd. 28, Nr. 1, S. 75–105, 2004.
- [6] K. Peffers, T. Tuunanen, M. A. Rothenberger, und S. Chatterjee, “A Design Science Research Methodology for Information Systems Research”, *Journal of Management Information Systems*, Bd. 24, Nr. 3, S. 45–77, 2007.
- [7] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, *Advances in Neural Information Processing Systems*, Bd. 33, S. 9459–9474, 2020.
- [8] L. Huang et al., “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions”, *ACM Transactions on Information Systems*, 2024. doi: 10.1145/3703155.
- [9] OWASP, “LLM01:2025 Prompt Injection - OWASP Gen AI Security Project”, 2025. [Online]. Verfügbar unter: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>

- [10] Anonymous, “Securing AI Agents Against Prompt Injection Attacks”, *arXiv preprint arXiv:2511.15759*, 2025.
- [11] L. Zheng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena”, *Advances in Neural Information Processing Systems*, Bd. 36, S. 46595–46623, 2023.
- [12] K. Greshake et al., “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection”, *arXiv preprint arXiv:2302.12173*, 2023.
- [13] Vectara Inc., “Benchmarking LLM Faithfulness in RAG with Evolving Leaderboards”, *arXiv preprint arXiv:2505.04847*, 2025.
- [14] Y. Liu et al., “G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment”, in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [15] Lakera AI, “Indirect Prompt Injection: The Hidden Threat Breaking Modern AI Systems”, 2025. [Online]. Verfügbar unter: <https://www.lakera.ai/blog/indirect-prompt-injection>